0043. Multiply Strings

dium | Category: Math & Geometry | LeetCode: https://leetcode.com/problems/mu

Problem

- Multiply two non-negative integers given as strings, without converting them to big integers.

Intuition

- Grade-school multiplication: digit i of num1 times digit j of num2 contributes to positions i + j and i + j + 1 of a result of length m + n.

Brute Force

- Repeated addition: hopelessly slow.

Optimal Approach

- Res = int[m + n].
- For i from m-1 down and j from n-1 down: sum = d1 d2 + res[i+j+1]; res[i+j+1] = sum % 10; res[i+j] += sum / 10.
- Skip leading zeros when building the string.

Worked Example

```
"123" x "45".

partial products placed by i+j:
3*5=15  3*4=12  2*5=10  2*4=8  1*5=5  1*4=4
res digits = 0 5 5 3 5
answer "5535"
```

Java Solution

```java
public class MultiplyStrings {
    public String multiply(String a, String b) {
        int m = a.length(), n = b.length();
        int[] res = new int[m + n];
        for (int i = m - 1; i >= 0; i--)
            for (int j = n - 1; j >= 0; j--) {
                int sum = (a.charAt(i) - '0') * (b.charAt(j) - '0') + res[i + j + 1];
                res[i + j + 1] = sum % 10;
                res[i + j] += sum / 10;
            }
        StringBuilder sb = new StringBuilder();
        for (int d : res) if (!(sb.length() == 0 && d == 0)) sb.append(d);
        return sb.length() == 0 ? "0" : sb.toString();
    }
}
```

Complexity

- Time: O(m n). Space: O(m + n).

Pitfalls

- If either input is "0", return "0", not an empty string.